

TUTORIAL COMPLETO · GUIDA PASSO-PASSO

Open WebUI Tor, I2P & Stealth Web Scraper

Un'infrastruttura di scraping a tre percorsi per AI locali:
Clearnet, Tor (Snowflake) e I2P, tutto in Docker.

Repository

github.com/thotganesh/openwebui-tor-i2p-scraper

Autore del progetto: thotganesh

Stack: Docker Compose · Playwright · FastAPI · Tor/Snowflake · i2pd · Open WebUI

Documento generato il 1 ottobre 2026 dallo stato attuale del branch main (17 commit)

Indice

1. Introduzione	3
1.1 Il progetto in numeri	3
1.2 Come leggere questa guida	3
2. Funzionalità principali	4
2.1 I tre percorsi a confronto	4
2.2 Perché Chromium per Clearnet e Firefox per Tor/I2P	4
3. Architettura	5
3.1 Cosa succede quando l'AI chiede una pagina	5
3.2 Porte e servizi	6
4. Prerequisiti	6
5. Quick Start	6
6. Tutorial passo-passo	7
6.1 Preparare le cartelle	7
6.2 tor-proxy: Tor + Snowflake	7
6.3 browser-tor: navigare con Tor	9
6.4 browser-clear: navigazione normale con stealth	14
6.5 browser-i2p: Garlic Routing con I2P	15
6.6 docker-compose.yml	17
6.7 Build e avvio	18
6.8 Verifica: la batteria di test	19
6.9 Integrazione con Open WebUI	19
6.10 Estrazione intelligente del testo	22
7. Sicurezza e OpSec	22
8. Risoluzione dei problemi	23
8.1 Tor non arriva a 100%	24
9. Comandi utili	24
10. Osservazioni sul repository	24
11. Cronologia del progetto	25
11.1 Le tappe principali	25
Appendice A · Glossario	26
Riferimenti	26

1. Introduzione

Open WebUI Tor, I2P & Stealth Web Scraper è un'infrastruttura Docker che dà a un modello AI locale (quello che usi con Open WebUI) la capacità di **leggere il web** in tre modi diversi: sulla rete normale (Clearnet), sulla rete anonima **Tor** e sulla rete **I2P**. Ogni percorso è un container indipendente con un browser reale (Playwright) che carica la pagina, ne estrae il contenuto utile e lo restituisce al modello come **Markdown pulito**.

Il progetto nasce da una serie di problemi pratici che si incontrano quando si collega un LLM a internet:

- **Pagine pesanti.** Immagini, font e video consumano RAM e, soprattutto, generano rumore che costa token. Qui i media vengono bloccati a livello di rete.
- **Siti con protezioni anti-bot.** Un semplice `requests.get()` viene fermato (es. da Cloudflare). Il browser Clearnet usa `playwright-stealth` per sembrare un utente reale.
- **Privacy e OpSec.** Per navigare in modo anonimo servono Tor e un browser configurato per non perdere informazioni (WebRTC, fingerprinting, geolocalizzazione...).
- **Provider che bloccano Tor.** Il trasporto **Snowflake** instrada il traffico Tor tramite volontari WebRTC, aggirando i blocchi.
- **Reti alternative.** Per i siti .i2p serve un router I2P (i2pd) e un browser isolato dalla rete normale.
- **Banner cookie e paywall.** Vengono rimossi prima dell'estrazione, così il testo che arriva al modello è solo il contenuto.

1.1 Il progetto in numeri

Elemento	Valore
Container Docker	5 (tor-proxy, browser-tor, browser-clear, i2p-proxy, browser-i2p)
Browser / motori	Chromium + stealth (Clearnet), Firefox hardened (Tor), Firefox hardened (I2P)
Tool per Open WebUI	3 (browse_web, browse_tor, browse_i2p)
Servizi FastAPI	3 file <code>app.py</code> , circa 350 righe di Python in totale
Commit sul branch main	17 (dal 28 al 30 settembre 2026)
Linguaggi / tecnologie	Python, JavaScript (iniettato), Docker, Go (build di Snowflake), shell

IMPORTANTE Questo documento descrive uno strumento tecnico di navigazione e di privacy. Usalo nel rispetto delle leggi del tuo Paese e dei termini di servizio dei siti che visiti. Anonimato non significa impunità: è tuo il compito di usare Tor e I2P per scopi legittimi (ricerca, privacy, accesso a informazioni censurate).

1.2 Come leggere questa guida

Se conosci già Docker, vai al **capitolo 5 (Quick Start)**: bastano pochi comandi. Se vuoi capire e ricostruire ogni pezzo da zero, segui il **capitolo 6 (Tutorial passo-passo)**, dove ogni file è mostrato per intero, con il codice reale preso dal repository e la spiegazione di ogni scelta. I capitoli 7-9 contengono sicurezza, risoluzione dei problemi e comandi utili. Il capitolo 10 raccoglie alcune osservazioni sul repository.

2. Funzionalità principali

Funzione	Descrizione
Tre percorsi di navigazione	Tre browser indipendenti: Clearnet, Tor e I2P, ciascuno nel proprio container.
OpSec dinamica e controllo JS	I tool Tor e I2P espongono il parametro <code>js_enabled</code> . L'AI può spegnere JavaScript su richiesta, riducendo fingerprinting e superficie d'attacco.
Cookie / GDPR killer	Un JavaScript iniettato individua e rimuove i banner di consenso (CMP) e alcuni paywall prima di estrarre il testo.
Estrazione intelligente	Mozilla Readability + <code>html2text</code> restituiscono solo l'articolo in Markdown, con un fallback per home page e motori di ricerca.
Ottimizzato per token e RAM	Immagini, video e font bloccati con il routing di Playwright (<code>block_media</code>).
Nuovo circuito Tor a richiesta	Prima di ogni navigazione, stem invia il segnale NEWNYM al ControlPort di Tor.
Trasporto Snowflake	Tor raggiunge la rete tramite Snowflake (WebRTC/ICE), utile quando l'ISP blocca Tor.
Credenziali fuori dal codice	Password e hash del ControlPort vengono letti dal file <code>.env</code> , mai scritti nel repository.

2.1 I tre percorsi a confronto

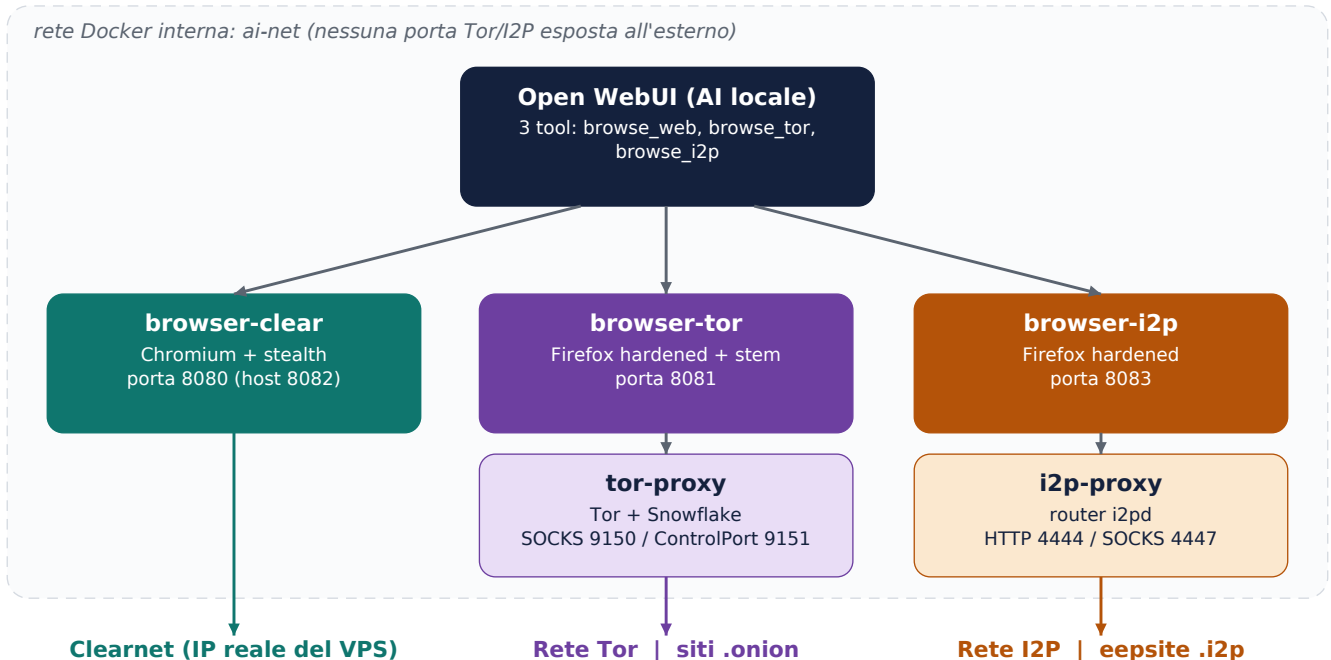
	browser-clear	browser-tor	browser-i2p
Motore	Chromium + playwright-stealth	Firefox hardened	Firefox hardened
Rete	Clearnet (IP reale)	Tor via Snowflake	I2P via i2pd (Garlic Routing)
Proxy usato	nessuno	socks5://tor-proxy:9150	http://i2p-proxy:4444
Porta interna / host	8080 / 127.0.0.1:8082	8081 / 127.0.0.1:8081	8083 / 127.0.0.1:8083
Parametri richiesta	url, block_media	url, block_media, js_enabled	url, block_media, js_enabled
WebRTC	attivo (come un Chrome vero)	disattivato (anti-leak)	disattivato
Timeout navigazione	60 s	90 s	120 s
Timeout del tool	90 s	120 s	150 s
Gestione errori	eccezione HTTP 500	eccezione HTTP 500	"airbag": risposta con messaggio chiaro

2.2 Perché Chromium per Clearnet e Firefox per Tor/I2P

Sulla rete normale conta somigliare a un utente reale, perciò si usa Chromium con le patch stealth e con WebRTC acceso (un Chrome vero ce l'ha sempre). Su Tor e I2P conta l'opposto: **essere indistinguibili e non perdere dati**. Firefox offre `privacy.resistFingerprinting` (la stessa tecnologia del Tor Browser) e la possibilità di impostare le preferenze di hardening direttamente da Playwright.

3. Architettura

Tutti i servizi vivono sulla rete Docker interna ai-net, condivisa con il container di Open WebUI. I browser sono esposti sull'host solo su 127.0.0.1, mentre i proxy Tor e I2P non pubblicano alcuna porta.



3.1 Cosa succede quando l'AI chiede una pagina

1. L'utente chiede qualcosa in chat; il modello decide quale tool chiamare (browse_web, browse_tor o browse_i2p) in base al system prompt.
2. Il tool (un piccolo file Python dentro Open WebUI) fa una POST su /browse del container corrispondente, passando url e opzioni.
3. Il servizio FastAPI avvia un browser headless con il proxy giusto (nessuno, SOCKS5 Tor, HTTP I2P). Per Tor chiede prima un nuovo circuito.
4. Playwright carica la pagina fino a domcontentloaded, attende 2,5 secondi, ed esegue il cookie killer se JavaScript è attivo.
5. L'HTML viene ripulito con Readability e convertito in Markdown con html2text. Se Readability non trova un articolo, si converte tutta la pagina.
6. Il servizio restituisce title, final_url e text; il tool tronca il testo a 4000 caratteri e lo passa al modello.

3.2 Porte e servizi

Servizio	Porta interna	Esposta sull'host	Note
tor-proxy	9150 (SOCKS), 9151 (ControlPort)	no	Mai pubblicare queste porte
browser-tor	8081	127.0.0.1:8081	Endpoint: /browse, /health, /webrtc-leak-check
browser-clear	8080	127.0.0.1:8082	Endpoint: /browse, /health
i2p-proxy	4444 (HTTP), 4447 (SOCKS)	no	Router i2pd, dati in ./i2p-data
browser-i2p	8083	127.0.0.1:8083	Endpoint: /browse

NOTA

Dentro Docker i servizi si parlano sulle porte **interne** (browser-clear:8080), non su quelle mappate sull'host (8082). I tool di Open WebUI devono quindi usare le porte interne.

4. Prerequisiti

- **Un VPS con Docker e Docker Compose** (comando `docker compose`).
- **UDP in uscita aperto.** Snowflake negozia via WebRTC/ICE, che usa UDP. Se il provider lo blocca, Tor resta fermo nel bootstrap.
- **Una rete Docker condivisa con Open WebUI**, chiamata qui `ai-net`. Se non esiste: `docker network create ai-net`.
- **Porte libere sull'host:** 8081 (browser-tor), 8082 (browser-clear), 8083 (browser-i2p).
- **Open WebUI già in esecuzione** e collegato a `ai-net`.
- **Un modello con function calling:** senza, l'AI non potrà chiamare i tool.
- Circa **2 GB** di spazio per l'immagine Playwright, più il tempo di compilazione di Snowflake al primo build.

5. Quick Start

Per chi conosce già Docker: lo stack completo in sette passi.

Comandi di installazione

```
# 1) Clona il repository
git clone https://github.com/thotganesh/openwebui-tor-i2p-scraper.git
cd openwebui-tor-i2p-scraper

# 2) Crea la rete Docker condivisa (salta se esiste già)
docker network create ai-net

# 3) Collega Open WebUI alla rete (salta se è già collegato)
docker network connect ai-net open-webui

# 4) Cartella dati I2P: i2pd gira come non-root e deve poter scrivere
mkdir -p i2p-data && chmod -R 777 i2p-data

# 5) Segreti: copia .env.example in .env e compila i valori
cp .env.example .env
# Genera l'hash della password del ControlPort:
docker run --rm debian:bookworm-slim bash -c \
  "apt-get update -qq && apt-get install -y -qq tor >/dev/null && tor --hash-password 'LaTuaPasswordSicura'"
# In .env: TOR_CONTROL_PASSWORD = password in chiaro, TOR_CONTROL_HASH = hash generato

# 6) Build e avvio
docker compose up -d --build

# 7) Attendi Tor: cerca "Bootstrapped 100% (done)"
docker compose logs -f tor-proxy
```

IMPORTANTE

Cambia la password di default. Il file `.env.example` contiene il valore segnaposto `change-me`: non lasciarlo. Dopo l'avvio, **i2pd** impiega 3-5 minuti al primo boot prima che i siti `.i2p` si risolvano. Il primo build richiede alcuni minuti (compilazione di `Snowflake` + immagine `Playwright` da circa 2 GB).

Dopo l'avvio installa i tre tool in Open WebUI (capitolo 6.9) ed esegui la batteria di test (capitolo 6.8).

6. Tutorial passo-passo

In questo capitolo ricostruiamo lo stack da zero. Ogni blocco di codice è il contenuto reale del file nel repository.

6.1 Preparare le cartelle

Creazione della struttura

```
mkdir -p ~/ai-stack/{tor-snowflake,browser-tor,browser-clear,browser-i2p,i2p-data}
# VITALE: i2pd gira come non-root e deve poter scrivere il suo addressbook
chmod -R 777 ~/ai-stack/i2p-data
cd ~/ai-stack
```

Struttura finale del progetto

```
~/ai-stack/
├── docker-compose.yml
├── .env                                (creato da .env.example, NON va committato)
├── tor-snowflake/
│   ├── Dockerfile
│   └── torrc
├── browser-tor/
│   ├── Dockerfile
│   ├── app.py
│   ├── entrypoint.sh                (self-check opzionale)
│   └── startup_check.py             (self-check opzionale)
├── browser-clear/
│   ├── Dockerfile
│   └── app.py
├── browser-i2p/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
├── openwebui-tools/
│   ├── web_browser_clear.py
│   ├── web_browser_tor.py
│   └── web_browser_i2p.py
└── i2p-data/                        (dati del router i2pd, ignorato da git)
```

NOTA

Il file `.gitignore` esclude `.env`, `i2p-data/` e `__pycache__`: segreti e dati del router non finiscono mai su GitHub.

6.2 tor-proxy: Tor + Snowflake

Questo container è il ponte verso la rete Tor. Usa **Snowflake** come pluggable transport: il client contatta un broker tramite domain fronting e poi negozia via WebRTC con un volontario che fa da proxy. Per l'ISP sembra normale traffico WebRTC.

6.2.1 Dockerfile multi-stage

tor-snowflake/Dockerfile

```
# --- Stage 1: build snowflake-client ---
FROM golang:1.23-bookworm AS builder

# Permette a Go di scaricare automaticamente la toolchain richiesta dal go.mod
ENV GOTOOLCHAIN=auto

RUN git clone https://git.torproject.org/pluggable-transport/snowflake.git /tmp/snowflake \
    && cd /tmp/snowflake/client \
    && go build -o /usr/local/bin/snowflake-client .

# --- Stage 2: immagine finale, solo runtime ---
FROM debian:bookworm-slim

RUN apt-get update && apt-get install -y \
    tor obfs4proxy ca-certificates \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /usr/local/bin/snowflake-client /usr/local/bin/snowflake-client
RUN chmod +x /usr/local/bin/snowflake-client

EXPOSE 9150
CMD ["tor", "-f", "/etc/tor/torrc"]
```

- **Stage 1 (builder):** compila snowflake-client da sorgente con Go.
- **Stage 2 (runtime):** parte da debian:bookworm-slim, installa solo tor, obfs4proxy e i certificati, e copia il binario compilato. L'immagine finale resta leggera.

NOTA

Il repository di Snowflake richiede una versione di Go molto recente. Con golang:1.23 + GOTOOLCHAIN=auto, Go scarica da solo la toolchain richiesta dal go.mod.

6.2.2 Password del ControlPort

Il ControlPort serve a chiedere a Tor un nuovo circuito (segnale NEWNYM) prima di ogni richiesta. È protetto da password: Tor vuole l'**hash**, il browser la password in **chiaro**.

Generazione dell'hash

```
docker run --rm debian:bookworm-slim bash -c \
    "apt-get update -qq && apt-get install -y -qq tor >/dev/null && tor --hash-password 'LaTuaPasswordSicura'"
# L'ultima riga stampata è l'hash, del tipo: 16:AF97...
```

.env.example → copialo in .env e compila

```
# Copy to .env and fill in. NEVER commit .env
TOR_CONTROL_PASSWORD=change-me
TOR_CONTROL_HASH=16:PASTE_THE_GENERATED_HASH_HERE
```

ATTENZIONE

Non committare mai il file .env. Il docker-compose.yml usa la sintassi \${VAR:?messaggio}: se una variabile manca, docker compose si ferma con un errore esplicito invece di avviarsi con una password vuota.

6.2.3 torrc

tor-snowflake/torrc

```
SocksPort 0.0.0.0:9150
UseBridges 1

ClientTransportPlugin snowflake exec /usr/local/bin/snowflake-client \
    -url https://snowflake-broker.torproject.net.global.prod.fastly.net/ \
    -front cdn.sstatic.net \
    -ice stun:stun.l.google.com:19302,stun:stun.antisip.com:3478,stun:stun.bluesip.net:3478,stun:stun.dus.net:3478,s
    tun:stun.epygi.com:3478,stun:stun.sonetel.com:3478,stun:stun.uls.co.za:3478,stun:stun.voipgate.com:3478,stun:s
    tun.voys.nl:3478

Bridge snowflake 192.0.2.3:80 2B280B23E1107BB62ABFC40DDCC8824814F80A72
    fingerprint=2B280B23E1107BB62ABFC40DDCC8824814F80A72
```

```
↪ url=https://snowflake-broker.torproject.net.global.prod.fastly.net/
↪ fronts=foursquare.com,github.githubassets.com ice=stun:stun.l.google.com:19302,stun:stun.antisp.com:3478,stun:s
↪ tun.bluesip.net:3478,stun:stun.dus.net:3478,stun:stun.epygi.com:3478,stun:stun.sonetel.com:3478,stun:stun.uls.co
↪ .za:3478,stun:stun.voipgate.com:3478,stun:stun.voys.nl:3478 utls-imitate=hellorandomizedalpn

ControlPort 0.0.0.0:9151
# HashedControlPassword is now injected via docker-compose and .env
```

- SocksPort 0.0.0.0:9150: i browser si collegano qui (solo dentro la rete Docker).
- UseBridges 1 + ClientTransportPlugin snowflake ...: Tor passa sempre da Snowflake.
- Bridge snowflake 192.0.2.3:80 ...: l'indirizzo 192.0.2.3 è un segnaposto della documentazione ufficiale; con Snowflake non conta, perché il client usa il broker e poi un proxy volontario.
- ControlPort 0.0.0.0:9151: usato da stem per NEWNYM.
- Nessun HashedControlPassword nel file: viene iniettato all'avvio dal compose (vedi 6.6).

ATTENZIONE

Le righe `-ice` e `Bridge` sono **una sola riga ciascuna**. Nel PDF vengono spezzate per ragioni di impaginazione (il simbolo ↪ indica la continuazione): nel file vero non vanno mai spezzate, tranne dove c'è il backslash.

6.3 browser-tor: navigare con Tor

Servizio FastAPI che pilota un Firefox headless attraverso il proxy SOCKS di tor-proxy. Ogni richiesta parte con un circuito Tor nuovo.

browser-tor/Dockerfile

```
FROM mcr.microsoft.com/playwright/python:v1.48.0-jammy

WORKDIR /app
RUN apt-get update && apt-get install -y libxml2-dev libxslt1-dev python3-dev && rm -rf /var/lib/apt/lists/*
RUN pip install fastapi uvicorn httpx playwright==1.48.0 stem readability-lxml html2text lxml
COPY app.py /app/app.py

EXPOSE 8081
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8081"]
```

L'immagine base è quella ufficiale Playwright per Python. Il pin `playwright==1.48.0` è importante: la versione del pacchetto deve coincidere con quella dei browser già presenti nell'immagine, altrimenti compare l'errore *Executable doesn't exist*. I pacchetti `libxml2-dev` e `libxslt1-dev` servono a compilare `lxml` (dipendenza di `Readability`).

6.3.1 Codice completo di app.py

browser-tor/app.py

```
import os
import asyncio
import socket
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from stem import Signal
from stem.control import Controller
from readability import Document
import html2text

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    wait_selector: str | None = None
    extract_text: bool = True
    block_media: bool = True
    js_enabled: bool = True

TOR_PROXY = {"server": "socks5://tor-proxy:9150"}
```

```

CONTROL_PASSWORD = os.environ.get("TOR_CONTROL_PASSWORD")

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,
    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "privacy.resistFingerprinting.letterboxing": True,
    "privacy.spoof_english": 2,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
    "media.navigator.enabled": False,
}

FIREFOX_PREFS_WEBRTC_TEST = dict(FIREFOX_PREFS)
FIREFOX_PREFS_WEBRTC_TEST["media.peerconnection.enabled"] = True

WEBRTC_LEAK_SCRIPT = """
async () => {
    if (typeof RTCPeerConnection === "undefined") return "DISABLED";
    return new Promise((resolve) => {
        const ips = new Set();
        const pc = new RTCPeerConnection({iceServers: [{urls: "stun:stun.l.google.com:19302"}]});
        pc.createDataChannel("");
        pc.onicecandidate = (e) => {
            if (!e.candidate) { resolve([...ips]); pc.close(); return; }
            const match = e.candidate.candidate.match(/(\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d{1,3})/);
            if (match) ips.add(match[1]);
        };
        pc.createOffer().then(o => pc.setLocalDescription(o));
        setTimeout(() => { resolve([...ips]); pc.close(); }, 5000);
    });
}
"""

COOKIE_KILLER_JS = """
async () => {
    const KNOWN_CMP_SELECTORS = [
        '#onetrust-banner-sdk', '#onetrust-consent-sdk',
        '#cookiebot', '#CybotCookiebotDialog',
        '.qc-cmp2-container', '#qc-cmp2-container',
        '#truste-consent-track', '.truste_box_overlay',
        '.cc-window', '.fc-consent-root', '#cookie-notice',
        '.paywall', '[class*="cookie-banner"]', '[id*="cookie-consent"]'
    ];
    KNOWN_CMP_SELECTORS.forEach(sel => {
        document.querySelectorAll(sel).forEach(el => el.remove());
    });
    document.body.style.overflow = 'auto';
}
"""

def request_new_circuit():
    try:
        tor_proxy_ip = socket.gethostbyname("tor-proxy")
        with Controller.from_port(address=tor_proxy_ip, port=9151) as controller:
            controller.authenticate(password=CONTROL_PASSWORD)
            controller.signal(Signal.NEWNYM)
            print(f"[info] Nuovo circuito richiesto con successo (via {tor_proxy_ip})")
    except Exception as e:
        print(f"[warn] Impossibile richiedere nuovo circuito: {e}")

def pulisci_testo(html_content: str) -> str:
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0

    try:
        doc = Document(html_content)
        testo_pulito = h.handle(doc.summary()).strip()
        if testo_pulito:
            return testo_pulito
    except Exception:
        pass

    try:
        return h.handle(html_content).strip()
    except Exception as e:

```

```

        return f"Errore nell'estrazione del testo: {e}"

@app.post("/browse")
async def browse(req: BrowseRequest):
    request_new_circuit()
    await asyncio.sleep(1)

    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=TOR_PROXY,
            firefox_user_prefs=FIREFOX_PREFS,
        )
        context = await browser.new_context(
            viewport={"width": 1000, "height": 900},
            timezone_id="UTC",
            locale="en-US",
            java_script_enabled=req.js_enabled,
        )
        page = await context.new_page()

        if req.block_media:
            await page.route("**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}", lambda route: route.abort())

        await page.goto(req.url, wait_until="domcontentloaded", timeout=90000)

        await page.wait_for_timeout(2500)

        # Only execute the Cookie Killer if JS is enabled!
        if req.js_enabled:
            try:
                await page.evaluate(COOKIE_KILLER_JS)
            except Exception as e:
                print(f"[warn] Failed to execute Cookie Killer: {e}")

        result = {"title": await page.title(), "final_url": page.url}
        if req.extract_text:
            result["text"] = pulisci_testo(await page.content())

        await browser.close()
        return result

@app.get("/webrtc-leak-check")
async def webrtc_leak_check():
    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=TOR_PROXY,
            firefox_user_prefs=FIREFOX_PREFS_WEBRTC_TEST,
        )
        page = await (await browser.new_context()).new_page()
        await page.goto("data:text/html,<html><body></body></html>")
        leaked_ips = await page.evaluate(WEBRTC_LEAK_SCRIPT)
        await browser.close()
        return {
            "note": "Test con WebRTC volutamente riabilitato. In produzione (/browse) resta sempre disattivato.",
            "leaked_ips": leaked_ips,
            "leak_detected": len(leaked_ips) > 0,
        }

@app.get("/health")
async def health():
    return {"status": "ok"}

```

6.3.2 Spiegazione blocco per blocco

Blocco	Cosa fa
BrowseRequest	Modello Pydantic della richiesta: <i>url</i> , <i>wait_selector</i> (non usato nel flusso), <i>extract_text</i> , <i>block_media</i> e <i>js_enabled</i> , tutti con default sicuri.
TOR_PROXY, CONTROL_PASSWORD	Indirizzo SOCKS5 del container Tor e password del ControlPort letta dalla variabile d'ambiente (arriva dal .env).
FIREFOX_PREFS	Hardening di Firefox: vedi tabella 6.3.3.
WEBRTC_LEAK_SCRIPT	JavaScript di diagnosi: prova a creare una <code>RTCPeerConnection</code> e raccoglie gli IP dei candidati ICE. Se WebRTC è disattivato risponde "DISABLED".

Blocco	Cosa fa
COOKIE_KILLER_JS	Rimuove dal DOM i contenitori dei CMP più comuni (OneTrust, Cookiebot, Quantcast, TrustArc...) e ripristina lo scroll.
request_new_circuit()	Risolve <i>tor-proxy</i> in IP (stem vuole un IP letterale), si autentica sul ControlPort 9151 e invia Signal.NEWNYM. Se fallisce stampa un warning ma non blocca la navigazione.
pulisci_testo()	Estrazione a due livelli: Readability.summary() → html2text; se vuoto, html2text su tutto l'HTML.
POST /browse	Nuovo circuito → pausa 1 s → Firefox con proxy e prefs → contesto con viewport 1000x900, fuso UTC e lingua en-US → blocco media opzionale → goto (domcontentloaded, 90 s) → attesa 2,5 s → cookie killer (solo se JS attivo) → estrazione.
GET /webrtc-leak-check	Endpoint diagnostico: avvia Firefox con WebRTC volutamente riacceso e mostra quali IP trapelerebbero. Dimostra perché in produzione deve restare spento.
GET /health	Risponde {"status": "ok"} per i controlli di raggiungibilità.

6.3.3 Le preferenze di hardening di Firefox

Preferenza	Effetto
media.peerconnection.enabled = False	Disattiva WebRTC: evita che l'IP reale trapeli via ICE/STUN aggirando il SOCKS.
network.dns.disablePrefetch = True	Niente prefetch DNS (richieste che potrebbero uscire dal percorso previsto).
network.prefetch-next = False	Niente precaricamento dei link successivi.
geo.enabled = False	Geolocalizzazione disattivata.
privacy.resistFingerprinting = True	Uniforma valori come risoluzione, fuso orario e user agent, riducendo l'unicità del browser.
...resistFingerprinting.letterboxing = True	La finestra usa dimensioni arrotondate (solo browser-tor).
privacy.spoof_english = 2	Impone la lingua inglese (solo browser-tor).
webgl.disabled = True	WebGL spento: è una fonte di fingerprint della GPU.
dom.battery.enabled = False	Nessun accesso allo stato della batteria.
device.sensors.enabled = False	Sensori del dispositivo non accessibili.
media.navigator.enabled = False	Niente API per camera/microfono (solo browser-tor).

6.3.4 Perché un nuovo circuito a ogni richiesta

Playwright non supporta l'autenticazione user/password su SOCKS5, quindi non si può usare la tecnica "credenziali casuali = circuito isolato". Si ricorre al ControlPort: NEWNYM chiede a Tor di usare circuiti nuovi per le connessioni successive. Tor limita il segnale a circa uno ogni 10 secondi, quindi due richieste ravvicinate possono condividere lo stesso IP di uscita.

6.3.5 Self-check all'avvio (opzionale)

Nel repository ci sono due file aggiuntivi, `entrypoint.sh` e `startup_check.py`, pensati come "cintura di sicurezza": prima di avviare il servizio verificano che WebRTC sia davvero disattivato e, se rilevano un leak, **rifiutano di avviare il container**.

browser-tor/entrypoint.sh

```
#!/bin/bash
set -e

echo "[entrypoint] Attendo che tor-proxy sia raggiungibile..."
for i in $(seq 1 30); do
    if curl -s -m 2 --socks5-hostname tor-proxy:9150 https://check.torproject.org/api/ip > /dev/null 2>&1; then
        echo "[entrypoint] tor-proxy raggiungibile."
        break
    fi
    sleep 2
done

python3 /app/startup_check.py
if [ $? -ne 0 ]; then
    echo "[entrypoint] Self-check fallito. Uscita."
    exit 1
fi

echo "[entrypoint] Self-check superato, avvio uvicorn..."
exec uvicorn app:app --host 0.0.0.0 --port 8081
```

browser-tor/startup_check.py

```
import asyncio
import sys
from playwright.async_api import async_playwright

TOR_PROXY = {"server": "socks5://tor-proxy:9150"}

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,
    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "privacy.resistFingerprinting.letterboxing": True,
    "privacy.spoof_english": 2,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
    "media.navigator.enabled": False,
}

WEBRTC_LEAK_SCRIPT = """
async () => {
    if (typeof RTCPeerConnection === "undefined") return "DISABLED";
    return new Promise((resolve) => {
        const ips = new Set();
        const pc = new RTCPeerConnection({iceServers: [{urls: "stun:stun.l.google.com:19302"}]});
        pc.createDataChannel("");
        pc.onicecandidate = (e) => {
            if (!e.candidate) { resolve([...ips]); pc.close(); return; }
            const match = e.candidate.candidate.match(/(\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d{1,3})/);
            if (match) ips.add(match[1]);
        };
        pc.createOffer().then(o => pc.setLocalDescription(o));
        setTimeout(() => { resolve([...ips]); pc.close(); }, 5000);
    });
}
"""

async def main():
    print("[startup-check] Verifica configurazione WebRTC di produzione...", flush=True)
    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=TOR_PROXY,
            firefox_user_prefs=FIREFOX_PREFS,
        )
        page = await (await browser.new_context()).new_page()
```

```

        await page.goto("data:text/html,<html><body></body></html>")
        result = await page.evaluate(WEBRTC_LEAK_SCRIPT)
        await browser.close()

    if result == "DISABLED":
        print("[startup-check] OK: RTCPeerConnection non disponibile, nessun rischio leak.", flush=True)
        sys.exit(0)

    if isinstance(result, list) and len(result) > 0:
        print(f"[startup-check] ERRORE CRITICO: WebRTC leak rilevato! IP esposti: {result}", flush=True)
        print("[startup-check] Il container NON si avvia per motivi di sicurezza.", flush=True)
        sys.exit(1)

    print("[startup-check] OK: nessun leak rilevato.", flush=True)
    sys.exit(0)

if __name__ == "__main__":
    asyncio.run(main())

```

NOTA

Nel Dockerfile attuale il CMD avvia direttamente uvicorn e questi due file non vengono copiati nell'immagine: sono quindi un'aggiunta pronta all'uso ma **non ancora collegata**. Per attivarli servono tre modifiche, descritte nel capitolo 10.

6.4 browser-clear: navigazione normale con stealth

Qui l'obiettivo è opposto a Tor: **sembrare un utente comune** e superare le protezioni anti-bot, usando il tuo IP reale.

browser-clear/Dockerfile

```

FROM mcr.microsoft.com/playwright/python:v1.48.0-jammy

WORKDIR /app
RUN apt-get update && apt-get install -y libxml2-dev libxslt1-dev python3-dev && rm -rf /var/lib/apt/lists/*
RUN pip install fastapi uvicorn playwright-stealth httpx playwright==1.48.0 readability-lxml html2text lxml
COPY app.py /app/app.py

EXPOSE 8080
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]

```

browser-clear/app.py

```

from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from playwright_stealth import Stealth
from readability import Document
import html2text

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    wait_selector: str | None = None
    extract_text: bool = True
    block_media: bool = True

COOKIE_KILLER_JS = """
async () => {
    const KNOWN_CMP_SELECTORS = [
        '#onetrust-banner-sdk', '#onetrust-consent-sdk',
        '#cookiebot', '#CybotCookiebotDialog',
        '.qc-cmp2-container', '#qc-cmp2-container',
        '#truste-consent-track', '.truste_box_overlay',
        '.cc-window', '.fc-consent-root', '#cookie-notice',
        '.paywall', '[class*="cookie-banner"]', '[id*="cookie-consent"]'
    ];
    KNOWN_CMP_SELECTORS.forEach(sel => {
        document.querySelectorAll(sel).forEach(el => el.remove());
    });
    document.body.style.overflow = 'auto';
}
"""

```

```

def pulisci_testo(html_content: str) -> str:
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0

    try:
        # Tentativo 1: Cerca un articolo strutturato
        doc = Document(html_content)
        testo_pulito = h.handle(doc.summary()).strip()
        if testo_pulito:
            return testo_pulito
    except Exception:
        pass

    try:
        # Fallback: Se è una home page/motore di ricerca, formatta tutto
        return h.handle(html_content).strip()
    except Exception as e:
        return f"Errore nell'estrazione del testo: {e}"

@app.post("/browse")
async def browse(req: BrowseRequest):
    async with Stealth().use_async(async_playwright()) as p:
        browser = await p.chromium.launch(headless=True)
        context = await browser.new_context(
            user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
            ↪ Chrome/126.0 Safari/537.36",
            viewport={"width": 1366, "height": 768},
        )
        page = await context.new_page()

        if req.block_media:
            await page.route("**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}", lambda route: route.abort())

        await page.goto(req.url, wait_until="domcontentloaded", timeout=60000)

        await page.wait_for_timeout(2500)
        await page.evaluate(COOKIE_KILLER_JS)

        result = {"title": await page.title(), "final_url": page.url}
        if req.extract_text:
            result["text"] = pulisci_testo(await page.content())

        await browser.close()
        return result

@app.get("/health")
async def health():
    return {"status": "ok"}

```

- **Stealth:** `Stealth().use_async(async_playwright())` (API di playwright-stealth 2.x) maschera i segnali tipici dell'automazione.
- **User agent e viewport realistici:** Chrome 126 su Windows, 1366x768.
- **WebRTC lasciato acceso:** un Chrome vero ce l'ha sempre, e qui l'IP reale non è un segreto.
- **block_media attivo di default** per velocità e risparmio di RAM. Con `false` carica tutto come un utente vero: utile contro anti-bot aggressivi, perché un Chrome che non carica mai le immagini è un comportamento anomalo.

6.5 browser-i2p: Garlic Routing con I2P

I2P è una rete anonima separata da Tor, con i propri siti (*eepsite*, dominio `.i2p`) raggiungibili solo dall'interno. Il router `i2pd` costruisce i tunnel; il browser lo usa come proxy HTTP sulla porta 4444.

browser-i2p/Dockerfile

```
FROM mcr.microsoft.com/playwright/python:v1.40.0-jammy
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN playwright install firefox
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8083"]
```

browser-i2p/requirements.txt

```
fastapi
uvicorn
playwright
readability-lxml
html2text
pydantic
```

browser-i2p/app.py

```
import asyncio
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from readability import Document
import html2text

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    extract_text: bool = True
    block_media: bool = True
    js_enabled: bool = True

# i2pd espone il proxy HTTP sulla porta 4444
I2P_PROXY = {"server": "http://i2p-proxy:4444"}

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,
    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
}

COOKIE_KILLER_JS = """
async () => {
    const KNOWN_CMP_SELECTORS = [
        '#onetrust-banner-sdk', '.cc-window', '.qc-cmp2-container',
        '[class*="cookie-banner"]', '[id*="cookie-consent"]'
    ];
    KNOWN_CMP_SELECTORS.forEach(sel => {
        document.querySelectorAll(sel).forEach(el => el.remove());
    });
    document.body.style.overflow = 'auto';
}
"""

def pulisci_testo(html_content: str) -> str:
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0
    try:
        doc = Document(html_content)
        testo = h.handle(doc.summary()).strip()
        if testo: return testo
    except Exception:
        pass
    try:
        return h.handle(html_content).strip()
    except Exception as e:
        return f"Errore estrazione: {e}"

@app.post("/browse")
```

```

async def browse(req: BrowseRequest):
    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=I2P_PROXY,
            firefox_user_prefs=FIREFOX_PREFS,
        )
        context = await browser.new_context(
            viewport={"width": 1000, "height": 900},
            timezone_id="UTC",
            locale="en-US",
            java_script_enabled=req.js_enabled,
        )
        page = await context.new_page()

        if req.block_media:
            await page.route("**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}", lambda route: route.abort())

        # --- AIRBAG: Blocco try/except per gestire i siti offline o I2P non sincronizzato ---
        try:
            await page.goto(req.url, wait_until="domcontentloaded", timeout=120000)
            await page.wait_for_timeout(2500)
        except Exception as e:
            await browser.close()
            return {
                "title": "Errore di Rete I2P",
                "final_url": req.url,
                "text": f"Sito I2P attualmente irraggiungibile o router I2P non ancora sincronizzato. Dettaglio: {e}"
            }

        # Se il JS è abilitato, puliamo i popup
        if req.js_enabled:
            try:
                await page.evaluate(COOKIE_KILLER_JS)
            except Exception as e:
                print(f"[warn] Failed to execute Cookie Killer: {e}")

        result = {"title": await page.title(), "final_url": page.url}
        if req.extract_text:
            result["text"] = pulisci_testo(await page.content())

        await browser.close()
        return result

```

- **Garlic Routing:** tutto il traffico passa dal container i2p-proxy, mai dal tuo IP reale.
- **Controllo JavaScript:** js_enabled=false disattiva del tutto JS, consigliato su I2P contro fingerprinting e script ostili.
- **"Airbag":** il blocco try/except attorno a page.goto gestisce gli eepsite offline (frequentissimi) restituendo un messaggio comprensibile anziché un errore 500.
- **Timeout lungo (120 s):** I2P è intrinsecamente lento. Il tool lato Open WebUI usa 150 s e consiglia block_media=true.

6.6 docker-compose.yml

docker-compose.yml

```

services:
  tor-proxy:
    build: ./tor-snowflake
    container_name: tor-proxy
    networks: [ai-net]
    environment:
      - TOR_CONTROL_HASH=${TOR_CONTROL_HASH:?Set TOR_CONTROL_HASH in .env}
    command: ["sh", "-c", "exec tor -f /etc/tor/torrc --HashedControlPassword \"$$TOR_CONTROL_HASH\""]
    volumes:
      - ./tor-snowflake/torrc:/etc/tor/torrc:ro
    restart: unless-stopped

  browser-tor:
    build: ./browser-tor
    container_name: browser-tor
    networks: [ai-net]
    environment:

```

```

    - TOR_CONTROL_PASSWORD=${TOR_CONTROL_PASSWORD:?Set TOR_CONTROL_PASSWORD in .env}
    depends_on: [tor-proxy]
    ports:
      - "127.0.0.1:8081:8081"
    restart: unless-stopped

browser-clear:
  build: ./browser-clear
  container_name: browser-clear
  networks: [ai-net]
  ports:
    - "127.0.0.1:8082:8080"
  restart: unless-stopped

i2p-proxy:
  image: purplei2p/i2pd:latest
  container_name: i2p-proxy
  networks: [ai-net]
  command: --httpproxy.address=0.0.0.0 --socksproxy.address=0.0.0.0
  restart: unless-stopped
  volumes:
    - ./i2p-data:/home/i2pd/data

browser-i2p:
  build: ./browser-i2p
  container_name: browser-i2p
  networks: [ai-net]
  depends_on: [i2p-proxy]
  ports:
    - "127.0.0.1:8083:8083"
  restart: unless-stopped

networks:
  ai-net:
    external: true

```

- **Comando di tor-proxy:** `exec tor -f /etc/tor/torrc --HashedControlPassword "$$TOR_CONTROL_HASH"`. Il doppio \$\$ serve a far arrivare un singolo \$ alla shell del container, che legge la variabile al momento dell'avvio.
- **Variabili obbligatorie:** `${VAR:?...}` fa fallire il compose se mancano `TOR_CONTROL_HASH` o `TOR_CONTROL_PASSWORD`.
- **ai-net è external: true:** la rete esiste già ed è condivisa con Open WebUI.
- **Porte su 127.0.0.1:** i browser sono raggiungibili solo dall'host stesso. Non pubblicare mai le porte di Tor (9150/9151) né dei proxy I2P.
- **restart: unless-stopped** su ogni servizio: lo stack riparte da solo dopo un riavvio del VPS.
- **depends_on:** browser-tor aspetta tor-proxy, browser-i2p aspetta i2p-proxy (parte nell'ordine, ma non garantisce che la rete sia già pronta).

6.7 Build e avvio

Avvio dello stack

```

cd ~/ai-stack
docker compose build
docker compose up -d
docker compose ps          # tutti e cinque i container "Up", nessun restart loop
docker compose logs -f tor-proxy  # attendi "Bootstrapped 100% (done)"

```

IMPORTANTE

Warm-up di I2P: il router i2pd impiega circa 3-5 minuti al primo avvio per costruire i tunnel e popolare l'addressbook. Testa `http://stats.i2p` solo dopo questo tempo.

6.8 Verifica: la batteria di test

Esegui questi sette test in ordine. Usa sempre URL reali: un URL segnaposto restituisce un 404 che sembra un errore del codice.

Sette test di verifica

```
# 1) Il SOCKS di Tor funziona e l'IP è anonimo
docker run --rm --network ai-net curlimages/curl -s --socks5-hostname tor-proxy:9150
↳ https://check.torproject.org/api/ip

# 2) browser-clear: deve mostrare IsTor:false con l'IP reale del VPS
curl -s -X POST http://127.0.0.1:8082/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 3) browser-tor: deve mostrare IsTor:true con l'IP di un exit node
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 4) browser-tor su un vero .onion (DuckDuckGo)
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "http://duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion"}'

# 5) Rotazione del circuito: NEWNYM ha un rate limit di circa 10 s
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'; echo; sleep 12
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 6) Test diagnostico WebRTC (volutamente riacceso: deve mostrare il leak dell'IP reale)
curl -s http://127.0.0.1:8081/webrtc-leak-check

# 7) browser-i2p su un eepsite, JavaScript OFF (attendi ~5 min dopo il primo avvio)
curl -s -X POST http://127.0.0.1:8083/browse -H "Content-Type: application/json" \
  -d '{"url": "http://stats.i2p", "js_enabled": false}'
```

Test	Risultato atteso
SOCKS diretto	{"IsTor":true,"IP":"..."}
browser-clear	IsTor: false con l'IP reale del VPS
browser-tor	IsTor: true, IP di uscita diverso da quello del VPS
browser-tor su .onion	HTTP 200 con titolo e testo della pagina
Rotazione circuito	IP di uscita diversi tra le due richieste
webrtc-leak-check	leaked_ips con l'IP reale: prova che WebRTC deve restare disattivato
browser-i2p su stats.i2p	HTTP 200 con titolo e testo dell'eepsite (dopo il warm-up)

6.9 Integrazione con Open WebUI

6.9.1 Rete e raggiungibilità

Verifica che Open WebUI raggiunga i browser

```
docker inspect open-webui --format '{{json .NetworkSettings.Networks}}' # ai-net deve comparire
docker network connect ai-net open-webui # solo se manca
docker exec open-webui curl -s -m 10 http://browser-clear:8080/health # {"status":"ok"}
docker exec open-webui curl -s -m 10 http://browser-tor:8081/health # {"status":"ok"}
```

6.9.2 Installazione dei tre tool

1. In Open WebUI vai su **Admin → Workspace → Tools → nuovo tool**.
2. Incolla il contenuto di **un file per volta** (uno per tool) dalla cartella openwebui-tools/.
3. Salva e attiva **tutti e tre** i tool nella chat; assicurati che il modello supporti il function calling.

IMPORTANTE

Tutti e tre i tool sono necessari: sono i tre pilastri dello stesso stack. Il tool Clear parla con browser-clear, il tool Tor con browser-tor e il tool I2P con browser-i2p.

Tool 1 · web_browser_clear.py

```
"""
title: Web Browser (Clear)
description: Naviga siti web normali tramite browser headless Chromium con stealth anti-detection
"""
import httpx

class Tools:
    def __init__(self):
        self.base_url = "http://browser-clear:8080"

    async def browse_web(self, url: str) -> str:
        """
        Naviga una pagina web normale ed estrae il testo.
        :param url: URL completo da visitare (es. https://example.com)
        """
        async with httpx.AsyncClient(timeout=90) as client:
            r = await client.post(f"{self.base_url}/browse", json={"url": url})
            data = r.json()
            return f"Titolo: {data.get('title')}\nURL finale: {data.get('final_url')}\n\n{data.get('text',
            ↳ ''[:4000])}"
```

Tool 2 · web_browser_tor.py

```
"""
title: Web Browser (Tor/Anonymous)
description: Naviga siti .onion o normali in forma completamente anonima tramite rete Tor (Snowflake). Permette il
↳ controllo del JavaScript per azzerare il fingerprinting.
"""

import httpx

class Tools:
    def __init__(self):
        self.base_url = "http://browser-tor:8081"

    async def browse_tor(
        self, url: str, js_enabled: bool = True, block_media: bool = True
    ) -> str:
        """
        Naviga una pagina web (anche .onion) tramite la rete Tor per anonimato completo.
        :param url: URL completo da visitare, incluso indirizzi .onion.
        :param js_enabled: Imposta a False per disabilitare completamente JavaScript. Da usare quando l'utente
        ↳ chiede la MASSIMA SICUREZZA e anti-fingerprinting.
        :param block_media: Imposta a True per bloccare immagini/video pesanti e velocizzare Tor.
        """
        async with httpx.AsyncClient(timeout=120) as client:
            try:
                r = await client.post(
                    f"{self.base_url}/browse",
                    json={
                        "url": url,
                        "js_enabled": js_enabled,
                        "block_media": block_media,
                    },
                )
                r.raise_for_status()
                data = r.json()

                # Creiamo l'etichetta visiva per capire subito se JS era acceso o spento
                status_js = (
                    "JS: OFF (Stealth/Anonymized)" if not js_enabled else "JS: ON"
                )
                text_content = data.get("text", "")[:4000]
```

```

        return f"[TOR NETWORK | {status_js}]\nTitolo: {data.get('title')}\nURL finale:
        ↳ {data.get('final_url')}\n\n{text_content}"
    except Exception as e:
        return f"Errore durante la navigazione Tor: {str(e)}"

```

Tool 3 · web_browser_i2p.py

```

"""
title: Web Browser (I2P Darknet)
description: Naviga esclusivamente la rete chiusa I2P (siti .i2p) in totale sicurezza. Controllo JavaScript
↳ integrato per massima OpSec.
"""

import httpx

class Tools:
    def __init__(self):
        # Punta alla nuova porta 8083 del container browser-i2p
        self.base_url = "http://browser-i2p:8083"

    async def browse_i2p(
        self, url: str, js_enabled: bool = True, block_media: bool = True
    ) -> str:
        """
        Naviga siti della rete I2P (.i2p) tramite router i2pd locale (Garlic Routing).
        :param url: URL completo da visitare (deve essere un indirizzo .i2p).
        :param js_enabled: Imposta a False per disabilitare JavaScript. Consigliato per la massima sicurezza
        ↳ contro fingerprinting e script ostili sulla rete I2P.
        :param block_media: Imposta a True per bloccare immagini/video pesanti e velocizzare la navigazione I2P
        ↳ (che è intrinsecamente lenta).
        """
        async with httpx.AsyncClient(timeout=150) as client:
            try:
                r = await client.post(
                    f"{self.base_url}/browse",
                    json={
                        "url": url,
                        "js_enabled": js_enabled,
                        "block_media": block_media,
                    },
                )
                r.raise_for_status()
                data = r.json()

                status_js = "JS: OFF (Stealth)" if not js_enabled else "JS: ON"
                text_content = data.get("text", "")[:4000]

                return f"[I2P NETWORK | {status_js}]\nTitolo: {data.get('title')}\nURL finale:
                ↳ {data.get('final_url')}\n\n{text_content}"
            except Exception as e:
                return f"Errore durante la navigazione I2P (Il sito potrebbe essere offline, molto comune su I2P):
                ↳ {str(e)}"

```

NOTA

Il limite [:4000] sul testo tiene sotto controllo il consumo di token: aumentalo se ti servono estratti più lunghi. La docstring di ogni funzione è fondamentale: Open WebUI la usa per descrivere al modello cosa fa il tool e a cosa servono i parametri.

6.9.3 Evitare conflitti con la Web Search nativa

Se la Web Search integrata di Open WebUI è attiva, disattivala (**Admin Panel → Settings → Web Search → OFF**): con strumenti in competizione i modelli piccoli scelgono in modo incoerente. Poi inserisci questo nel System Prompt globale:

System prompt consigliato (traduzione italiana del testo del README)

```

Per navigare siti web normali usa SEMPRE il tool browse_web.
Per siti .onion, o quando l'utente chiede esplicitamente anonimato, usa SEMPRE il tool browse_tor.
Per eepsite .i2p usa SEMPRE il tool browse_i2p.
Non hai nessun altro modo di accedere a internet.
Tieni JavaScript attivo di default; disattivalo (js_enabled=false) solo quando l'utente chiede la massima
↳ sicurezza.

```

Se il modello non chiama i tool, controlla che siano attivi nella chat e che il modello abbia la capability di function calling (impostazioni del modello → Capabilities).

6.10 Estrazione intelligente del testo

Quattro meccanismi, presenti in tutti i browser:

1. **Blocco delle risorse (block_media):** `page.route("**/*.{png,jpg,...}")` ferma immagini, font e video prima del download. Pagine più veloci e meno memoria. Sui siti con anti-bot serio disattivalo.
2. **Cookie killer:** rimuove dal DOM i contenitori dei principali CMP e ripristina lo scorrimento. Non aggira i paywall lato server.
3. **Estrazione a due livelli:** Readability per l'articolo strutturato, poi fallback con `html2text` su tutto l'HTML (home page, indici, risultati di ricerca). Non si ottiene mai un errore secco.
4. `DOMContentLoaded` + **attesa fissa di 2500 ms** invece di `networkidle`: i siti pieni di pubblicità non raggiungono mai la "rete silenziosa" e finirebbero in timeout.

ATTENZIONE Alcuni CMP hanno protezioni anti-manomissione: su `repubblica.it` (Iubenda) rimuovere il banner con `el.remove()` ha svuotato l'intera pagina (l'HTML è passato da 496.335 a 15 caratteri). In questi casi conviene **nascondere** con `el.style.setProperty('display','none','important')` invece di rimuovere.

7. Sicurezza e OpSec

Le scelte di sicurezza del progetto, riassunte in un unico posto:

Area	Scelta nel progetto	Perché
Segreti	Password e hash nel file <code>.env</code> , ignorato da git; <code>\${VAR:?}</code> nel compose	Nessuna credenziale nel repository; avvio bloccato se mancano
Esposizione di rete	Porte dei browser solo su 127.0.0.1; proxy Tor e I2P senza porte pubblicate	Nessuno dall'esterno può usare i tuoi proxy anonimi
WebRTC	Disattivato su Tor e I2P; attivo solo nel test diagnostico	Con WebRTC acceso l'IP reale trapela via ICE/STUN
Fingerprinting	<code>resistFingerprinting</code> , WebGL off, viewport 1000x900, UTC, en-US	Rende il browser simile ad altri utenti Tor
JavaScript	Parametro <code>js_enabled</code> controllabile dall'AI	Massima sicurezza su darknet quando serve
Circuiti Tor	NEWNYM prima di ogni richiesta	Riduce la correlazione tra richieste diverse
Isolamento I2P	Container separato, proxy dedicato, nessun traffico clearnet	Nessuna esposizione dell'IP reale
Self-check	<code>startup_check.py</code> blocca il container se rileva un leak WebRTC	Fail-closed: meglio non partire che perdere l'IP

SUGGERIMENTO Il `chmod -R 777` sulla cartella `i2p-data` è il modo più rapido per far scrivere a `i2pd` (utente non-root). Su un server condiviso valuta un'alternativa più restrittiva: assegnare la cartella all'UID usato dal container, invece di concedere permessi di scrittura a tutti.

NOTA

Il contenuto delle pagine proviene da fonti non fidate. Un sito può contenere testo pensato per manipolare il modello (*prompt injection*). Tratta il testo restituito dai tool come dati, non come istruzioni, soprattutto se il tuo modello può eseguire altre azioni.

8. Risoluzione dei problemi

Errore / sintomo	Causa	Soluzione
I2P: container Restarting (139)	Permesso negato su /home/i2pd/data	<code>chmod -R 777 i2p-data</code>
I2P: NS_ERROR_UNKNOWN_PROXY_HOST	Il container i2p-proxy è giù o irraggiungibile	Controlla i permessi su i2p-data, poi <code>docker compose restart i2p-proxy</code>
I2P: Host not found in addressbook	Il router sta ancora facendo warm-up	Attendi ~5 minuti, poi prova <code>http://stats.i2p</code>
Tool I2P: errore o timeout	Eepsite offline (molto comune) o tunnel lenti	Riprova più tardi o prova un altro eepsite
Error parsing Bridge address 'auto'	Sintassi torrc errata o residua	Riscrivi il torrc da zero
general SOCKS server failure (obfs4)	Bridge obfs4 con fingerprint/cert inventati	Usa Snowflake con la riga Bridge ufficiale
invalid go version '1.25.12'	Go dell'immagine troppo vecchio	Multi-stage con go e ENV <code>GOTOOLCHAIN=auto</code>
pip3: not found	Immagine Playwright variante Node.js	Usa <code>mcr.microsoft.com/playwright/python</code>
No module named 'playwright'	L'immagine Python non include il pacchetto	<code>pip install playwright==1.48.0</code> esplicito
Executable doesn't exist ... chrome-headless-shell	Pacchetto Python più nuovo dei browser dell'immagine	Fissa <code>playwright==</code> alla versione dell'immagine
port is already allocated	Porta host già usata	Cambia la mappatura (es. <code>8082:8080</code>)
cannot import name 'stealth_async'	playwright-stealth 2.x ha una nuova API	<code>Stealth().use_async(async_playwright())</code>
RTCPeerConnection is not defined	Con WebRTC disattivato l'API non esiste	Test del leak con WebRTC riaccesso (endpoint dedicato)
Browser does not support socks5 proxy authentication	Playwright non supporta user/password su SOCKS5	Isola i circuiti con ControlPort e NEWNYM
Invalid IP address: tor-proxy	stem richiede un IP letterale	<code>socket.gethostbyname("tor-proxy")</code>
Document is empty / Unparseable	Readability non trova un articolo	Fallback <code>html2text</code> su tutto l'HTML
Pagina svuotata dopo il cookie killer	Il CMP rileva la rimozione del banner (anti-tamper)	Nascondi con <code>display:none</code> invece di <code>remove()</code>
Cannot read properties of null (reading 'style')	<code>document.body</code> assente durante un reload	<code>if (document.body) + try/except</code>
Banner cookie ancora nel testo	Selettori case-sensitive o CMP non in lista	Aggiungi gli ID del CMP del sito (o selettori con flag <code>i</code>)
Risposta 404 nei test	URL segnaposto preso alla lettera	Usa un URL reale

Errore / sintomo	Causa	Soluzione
curl risponde con 0 byte	Errore 500 nascosto da curl -s	curl -v e docker compose logs <servizio>

8.1 Tor non arriva a 100%

Se i log di tor-proxy restano fermi sul bootstrap, la causa più probabile è il **UDP in uscita bloccato** dal provider del VPS (Snowflake lo richiede). Verifica le regole del firewall del provider, poi riavvia con docker compose restart tor-proxy.

9. Comandi utili

Gestione quotidiana dello stack

```
cd ~/ai-stack
docker compose ps                # stato dei container
docker compose logs -f browser-tor  # log del browser Tor in tempo reale
docker compose logs -f i2p-proxy    # monitora i tunnel I2P
docker compose logs browser-clear --tail 30 | grep -i warn

# dopo aver modificato un app.py / Dockerfile:
docker compose build browser-clear browser-tor browser-i2p && docker compose up -d browser-clear browser-tor
~ browser-i2p

docker compose restart tor-proxy    # riavvia solo Tor
docker compose restart i2p-proxy    # riavvia solo I2P
docker compose down                 # ferma tutto (la rete ai-net resta)
```

10. Osservazioni sul repository

Leggendo il codice reale accanto al README sono emerse alcune differenze e possibili miglioramenti. Non sono errori gravi, ma conviene saperle prima di pubblicare il tutorial, così testi e codice restano coerenti.

#	Osservazione	Suggerimento
1	Self-check non attivo. entrypoint.sh e startup_check.py sono nel repo ma il Dockerfile di browser-tor non li copia e il CMD avvia direttamente uvicorn.	Aggiungi nel Dockerfile: COPY startup_check.py entrypoint.sh /app/, RUN chmod +x /app/entrypoint.sh e CMD ["/app/entrypoint.sh"]. Lo script usa curl: installalo con apt.
2	/health su browser-i2p. Il README (8.1) lo testa, ma browser-i2p/app.py non definisce l'endpoint.	Aggiungi @app.get("/health") come negli altri due servizi, oppure toglie il test dal README.
3	Cookie killer con remove(). Il README avverte che remove() svuota alcune pagine (es. lubenda), ma il codice usa ancora el.remove() nei tre servizi.	Sostituisci con el.style.setProperty('display', 'none', 'important') e ripristina lo scroll.
4	Tool Clear senza block_media. Nel README il tool ha il parametro block_media; il file web_browser_clear.py invece no (usa il default del server).	Allinea il file al README per poter disattivare il blocco sui siti anti-bot.
5	Piccole differenze nei Dockerfile. Il README mostra versioni semplificate (es. senza libxml2-dev, con EXPOSE 8083 su browser-i2p).	Copia i Dockerfile reali nel README, oppure rimanda ai file.
6	Versioni Playwright diverse. browser-tor e browser-clear usano 1.48.0, browser-i2p l'immagine 1.40.0 con un playwright senza pin in requirements.txt.	Fissa playwright==1.40.0 in requirements.txt (o allinea tutto a 1.48.0) per evitare discrepanze con i browser.

#	Osservazione	Suggerimento
7	Lingua mista. Commenti e stringhe sono in italiano, il README in inglese.	Scegli una lingua per il README o pubblica una versione italiana: questo PDF può fare da base.

11. Cronologia del progetto

Il progetto è cresciuto in due giornate, con 17 commit. Ecco la sequenza, che rispecchia anche l'ordine logico in cui costruire lo stack.

#	Data	Commit
1	28/09/2026	First release: Tor Snowflake + Stealth Scraper infrastructure
2	28/09/2026	Initialize README for Open WebUI Tor & Stealth Scraper
3	28/09/2026	Add complete tutorial for Tor and Snowflake setup
4	28/09/2026	Fix formatting and enhance README content
5	28/09/2026	Add web_browser_clear.py for web browsing functionality
6	28/09/2026	Add Tor web browser tool for anonymous browsing
7	28/09/2026	Document Open WebUI tools and installation steps
8	30/09/2026	Revise README for I2P integration and tool updates
9	30/09/2026	Revise README.md for clarity and completeness
10	30/09/2026	Add I2P web browser functionality
11	30/09/2026	Enhance browse_tor with JS and media control options
12	30/09/2026	Revise README for clarity and updated features
13	30/09/2026	Enhance README for I2P integration and features
14	30/09/2026	Add Quick Start section to README
15	30/09/2026	feat: Add I2P Garlic Routing and secure Tor credentials via .env
16	30/09/2026	Revise README for secrets configuration and Tor setup
17	30/09/2026	Clarify HashedControlPassword setup instructions

11.1 Le tappe principali

Fase	Cosa è stato fatto
28 set • Fondamenta	Infrastruttura Tor + Snowflake e scraper stealth; tutorial completo; primi tool Open WebUI (Clear e Tor).
30 set • Controllo e I2P	Tool Tor potenziato con controllo di JavaScript e media; aggiunta del browser I2P con Garlic Routing e del tool relativo.
30 set • Sicurezza	Password e hash del ControlPort spostati in .env; istruzioni chiare sull'hash; sezione Quick Start nel README.

Appendice A · Glossario

Termine	Significato
Open WebUI	Interfaccia web per modelli AI locali, estendibile con tool Python.
Tool (function calling)	Funzione che il modello può chiamare per ottenere informazioni o eseguire azioni.
Playwright	Libreria di Microsoft per pilotare browser (Chromium, Firefox, WebKit) da codice.
Headless	Browser che gira senza interfaccia grafica.
Tor	Rete di anonimato a circuiti (onion routing): il traffico passa da più nodi cifrati.
Snowflake	Pluggable transport di Tor che usa volontari WebRTC come ponti, utile dove Tor è bloccato.
Domain fronting	Tecnica che contatta un servizio tramite un dominio "di copertura" su una CDN.
WebRTC / ICE / STUN	Tecnologie per connessioni peer-to-peer nel browser; possono rivelare l'IP reale.
ControlPort / NEWNYM	Interfaccia di controllo di Tor; NEWNYM chiede nuovi circuiti per le connessioni successive.
stem	Libreria Python per controllare Tor tramite ControlPort.
.onion	Servizi nascosti raggiungibili solo tramite Tor.
I2P / i2pd	Rete anonima separata da Tor; <i>i2pd</i> è l'implementazione leggera del router in C++.
Eepsite	Sito web ospitato su I2P, dominio .i2p.
Garlic Routing	Variante dell'onion routing usata da I2P, con messaggi raggruppati ("spicchi d'aglio").
Fingerprinting	Identificare un utente dalle caratteristiche del suo browser (risoluzione, font, GPU...).
CMP	Consent Management Platform: i banner per cookie e consenso (OneTrust, Cookiebot...).
Readability	Algoritmo (Mozilla) che isola il contenuto principale di una pagina.
html2text	Libreria Python che converte HTML in Markdown.
OpSec	Operational Security: pratiche per non rivelare informazioni sensibili.
Prompt injection	Istruzioni nascoste in un contenuto web per manipolare il modello.

Riferimenti

- Repository: <https://github.com/thotganesh/openwebui-tor-i2p-scraper>
- Tor Project e Snowflake: <https://www.torproject.org>
- I2P / i2pd: <https://i2pd.website>
- Playwright per Python: <https://playwright.dev/python>
- Open WebUI: <https://openwebui.com>